

Global sensitivity analysis dla modelu SD Iranu metodami Morris i Sobol w Pythonie

Executive summary

Ten raport dostarcza **kompletnej, wykonalnej procedury global sensitivity analysis (GSA)** dla Twojego tygodniowego modelu system dynamics (SD) stabilności reżimu (zmienne: S/E/L/I/F/M/P/CPI/T/D) z metrykami wyjściowymi: **Tcrit**, **elite_fracture_prob**, **peak_protest**, **end_displaced_m**, **avg_loyal**. Model jest jawnie projektowany jako narzędzie badawcze do analizy „**resilience vs load**” w horyzoncie 1–5 lat oraz do testowania hipotez o kaskadach kryzysu między gospodarką (ropa/fiskus/inflacja), mobilizacją, represją i rdzeniem reżimu. [filecite\turn0file0](#)

Najważniejsze decyzje projektowe (kluczowe dla poprawnego GSA):

1. **Stochastyczność**: aby Morris i Sobol miały sens dla modelu losowego, wyniki dla pojedynczego punktu parametrów muszą być uśrednione po replikacjach (różne seedy) albo trzeba użyć „common random numbers” (stały zestaw seedów dla wszystkich próbek). Literatura podkreśla, że klasyczne GSA było rozwijane głównie dla deterministycznych symulatorów, a dla stochastycznych potrzebne są modyfikacje / meta-modele / estymacja na średnich lub innych funkcjonalach rozkładu wyjścia. [1](#)
2. **Koszt obliczeniowy**:
3. Morris jest metodą przesiewową (screening) i jest relatywnie tani; SALib zwraca m.in. μ^* , σ oraz przedziały ufności μ^*_conf . [2](#)
4. Sobol wymaga znacznie więcej ewaluacji: przy D parametrach SALib generuje **$N \cdot (D+2)$** próbek (bez drugiego rzędu) albo **$N \cdot (2D+2)$** (z interakcjami S_2), co wprost determinuje budżet obliczeń. [3](#)
5. **Wyniki dyskretne** (0/1) typu elite_fracture: do Sobola lepiej używać **prawdopodobieństwa** (średnia z replikacji) zamiast pojedynczego losowego 0/1 (w przeciwnym razie wariancja jest zdominowana przez szum). To zgodne z logiką stochastycznych symulatorów i praktyką bootstrapowania/CI. [4](#)
6. Raport kończy się **praktycznymi rekomendacjami kalibracyjnymi dla Iranu** i listą źródeł danych do pobrania (makro, ropa, sankcje, protesty, internet shutdowns, uchodźcy). [5](#)

Kontekst badawczy i założenia metodyczne

Twoje pliki opisują model jako **minimalny, kalibrowalny szkielet SD** z tygodniowym krokiem, zestawem stoków i sprzężeń (ropa/fiskus $\rightarrow$ inflacja/stres $\rightarrow$ protest $\rightarrow$ stabilność/lojalność/elity) oraz procedurą Monte Carlo i metrykami wyjściowymi. [filecite\turn0file0](#) W „formalnym reżimie absolutnym” kluczowe jest rozdzielanie [FAKT]/[INFERENCJA]/[ZAŁOŻENIE] oraz jawne propagowanie niepewności. [filecite\turn0file1](#)

W GSA będziemy traktować 20 parametrów jako **[ZAŁOŻENIA] z rozkładami prior**, a wyniki metryk jako rozkłady wynikowe (po replikacjach). Procedura jest dwustopniowa:

- **Morris** → szybki screening: identyfikacja kilku parametrów dominujących oraz tych „martwych” (niewrażliwych). Metoda Morris i jej udoskonalenia (trajektorie optymalne) są standardem dla kosztownych modeli z wieloma wejściami. ⁶
- **Sobol** → dokładna dekompozycja wariancji: indeksy pierwszego rzędu S1, całkowite ST i (opcjonalnie) interakcje S2. SALib implementuje estymatory zgodne z literaturą (Saltelli i in. / Jansen dla ST). ⁷

Specyfikacja analizy: 20 parametrów, zakresy i rozkłady

Poniżej jest **priorytetowa lista 20 parametrów** (Twoja lista) wraz z **proponowanym zakresem** (bounds dla SALib) oraz **rozkładem** (transformacja z [0,1] do rozkładu docelowego). Uzasadnienia są krótkie i „badawcze”: chodzi o realistyczne priory dla Iranu, ale jednocześnie wystarczająco szerokie, by uchwycić bifurkacje. Model sam wskazuje kluczowe grupy do kalibracji: elastyczności ropy na sankcje/wojnę, pass-through inflacyjny, skuteczność represji/mobilizacji oraz wpływ kontroli informacji.

Konwencja:

- parametry „0–1 / intensywności” → **Beta** (skośne priory, bo skrajności są rzadsze),
- parametry dodatnie „rate” → **truncated normal** wokół wartości bazowej, żeby utrzymać masę w pobliżu domyślnych założeń, ale dopuścić ogony.

Tabela priory (parametr, zakres, rozkład, uzasadnienie)

parametr	zakres (min–max)	rozkład (propozycja)	uzasadnienie priory
k_san_oil	0.20–0.90	Beta(5,3) skalowana	elastyczność eksportu ropy na sankcje; wysoka, ale nie zawsze bliska 1; model traktuje to jako parametr 1. rzędu
k_war_oil	0.30–0.95	Beta(4,4) skalowana	zakłócenie eksportu/produkcji w warunkach wojny; szeroki zakres scenariuszowy
k_pi_san	0.00–0.50	TruncNormal($\mu=0.25$, $\sigma=0.06$)	pass-through sankcji do inflacji; źródła kalibracji: inflacja CBI/WDI oraz reżimy sankcyjne OFAC/UE
k_pi_war	0.00–0.60	TruncNormal($\mu=0.35$, $\sigma=0.08$)	pass-through wojny do inflacji (logistyka, import, waluta)
pi_base	0.10–0.80	TruncNormal($\mu=0.35$, $\sigma=0.10$)	Iran często ma chronicznie wysoką inflację; baza musi obejmować epizody ~10–70%+; dane referencyjne: WDI/CBI ⁸

parametr	zakres (min–max)	rozkład (propozycja)	uzasadnienie priory
r0	0.50–1.20	TruncNormal($\mu=0.90$, $\sigma=0.10$)	„bazowa” zdolność represji; Iran ma bardzo restrykcyjne środowisko online i silny aparat, ale parametr jest proxy ⁹
k_mob	0.02–0.15	TruncNormal($\mu=0.08$, $\sigma=0.02$)	szybkość mobilizacji (stock P); dostraja falowość protestów vs „lawiny”
k_demob	0.02–0.20	TruncNormal($\mu=0.10$, $\sigma=0.03$)	szybkość demobilizacji (represja/zmęczenie); istotne w protest–represja nexus filecite turn0file0
k_loss_p	0.02–0.20	TruncNormal($\mu=0.08$, $\sigma=0.03$)	jak szybko protest eroduje stabilność; parametr progowy dla Tcrit
k_cons	0.01–0.15	TruncNormal($\mu=0.06$, $\sigma=0.02$)	„konsensus rdzenia” (E·L·F) jako zasób odporności (resilience) filecite turn0file0
k_pay	0.00–0.18	TruncNormal($\mu=0.05$, $\sigma=0.03$)	transfer fiskalny → lojalność aparatu; trudny do twardej kalibracji (latent) filecite turn0file1
k_loss_l	0.00–0.25	TruncNormal($\mu=0.06$, $\sigma=0.03$)	erozja lojalności przez wojnę i stres ekonomiczny (proxy)
k_split	0.00–0.20	TruncNormal($\mu=0.05$, $\sigma=0.03$)	rozszczenie aparatu, gdy elity pękają; kluczowy mechanizm „rdzeń vs elity” filecite turn0file1
k_elite_cost	0.00–0.20	TruncNormal($\mu=0.05$, $\sigma=0.03$)	koszty/stres → erozja spójności elit (latent) filecite turn0file1
k_elite_protest	0.00–0.20	TruncNormal($\mu=0.04$, $\sigma=0.03$)	protest → erozja elit; w autokracji to kanał destabilizacji 2. konfliktu (elity) filecite turn0file1
k_info_emerg	0.00–0.20	TruncNormal($\mu=0.04$, $\sigma=0.03$)	„awaryjna” eskalacja kontroli informacji w kryzysie; Iran ma udokumentowane shutdowny ¹⁰
k_info	0.00–0.08	TruncNormal($\mu=0.02$, $\sigma=0.015$)	wpływ I na stabilność; mały parametr, ale może mieć efekt progowy przez sprzężenia filecite turn0file0
k_rev	0.02–0.30	TruncNormal($\mu=0.12$, $\sigma=0.06$)	dochód ropny → odbudowa fiskusa; buforuje kryzys (w modelu kluczowy) filecite turn0file0
k_war_spend	0.02–0.25	TruncNormal($\mu=0.10$, $\sigma=0.05$)	drenaż fiskusa przez wojnę; zwiększa presję na L/E i inflację

parametr	zakres (min–max)	rozkład (propozycja)	uzasadnienie priory
k_rally_s	0.00–0.20	TruncNormal($\mu=0.05$, $\sigma=0.05$)	krótkookresowy „rally-around-the-flag” dla stabilności; wygasa w czasie (tau) filecite turn0file0

Uwagi implementacyjne: - SALib wymaga **bounds**; rozkłady realizujemy przez **transformację** próbek z [0,1] do rozkładów docelowych (inverse CDF), bazując na SciPy.

- Jeżeli zechcesz uwzględnić zależności (np. korelacja: k_san_oil ↔ k_pi_san), klasyczny Sobol wymaga niezależności wejść; zależności to temat osobny (np. generalized Sobol / Shapley effects). ¹¹

Interfejs modelu i przygotowanie środowiska uruchomieniowego

Twoja dokumentacja już zawiera kompletny szkielet kodu SD i metryk, z tygodniowym krokiem, scenariuszami exo i funkcją compute_metrics. filecite turn0file0 Poniżej buduję to w formie pakietu „GSA-ready”.

Struktura katalogu (proponowana)

```
sd_iran_gsa/
  README.md
  requirements.txt
  config.yaml
  sd_iran_model.py
  model_interface.py
  gsa_common.py
  run_morris.py
  run_sobol.py
  run_local_sa.py
  run_extremes.py
  notebooks/
    00_template_gsa.ipynb
  outputs/
    (tu zapiszesz CSV/PNG)
```

Wymagane biblioteki i wersje

- SALib (dostarcza Morris i Sobol; wersja stabilna 1.5.2, Python ≥ 3.9). ¹²
- NumPy / SciPy / pandas / matplotlib (wymagane również w SALib). ¹³
- joblib albo Dask do równoległości (zalecane, bo Sobol robi dziesiątki tysięcy uruchomień; joblib jest prosty dla „embarrassingly parallel”). ¹⁴

Minimalny requirements.txt

```
SALib==1.5.2
numpy>=1.26
scipy>=1.11
```

```
pandas>=2.1
matplotlib>=3.8
joblib>=1.3
pyyaml>=6.0
tqdm>=4.66
```

config.yaml (scenariusz, horyzont, replikacje)

```
scenario: "impas"
# "szybka_wojna" | "dlugotrwała_wojna" | "impas" | "eskalacja_regionalna"
years: 3
seed_base: 12345

# stochastyczność: ile replikacji per punkt parametrów
n_reps: 5
rep_seed_stride: 100000    # seed_i = seed_base + rep_seed_stride * i +
offset

# progi metryk
stab_crit: 0.30
elite_crit: 0.35
elite_streak_weeks: 4

# GSA: Morris
morris_num_levels: 8
morris_trajectories: 30
morris_optimal_trajectories: 10    # Campolongo enhancement
morris_local_optimization: true

# GSA: Sobol
sobol_N: 1024
sobol_calc_second_order: false
sobol_num_resamples: 1000
sobol_conf_level: 0.95

# równoległość
n_jobs: -1    # joblib: -1 = wszystkie rdzenie
```

Kod modelu: sd_iran_model.py

Poniżej jest „przeniesiony” i uporządkowany model SD (zgodny z Twoim opisem i kodem referencyjnym), z `simulate()` i `compute_metrics()` w stylu Twojej dokumentacji.

```
# sd_iran_model.py
# -*- coding: utf-8 -*-

from __future__ import annotations
```

```

from dataclasses import dataclass
from typing import Dict, Optional, Tuple, List
import math
import numpy as np
import pandas as pd

def clip(x: float, lo: float = 0.0, hi: float = 1.0) -> float:
    return max(lo, min(hi, x))

def sigmoid(x: float) -> float:
    # zabezpieczenie przed overflow exp()
    x = max(-60.0, min(60.0, x))
    return 1.0 / (1.0 + math.exp(-x))

@dataclass
class Params:
    # --- Oil & fiscal ---
    oil_export_cap: float = 1.0
    k_san_oil: float = 0.6
    k_war_oil: float = 0.7
    k_rev: float = 0.12
    k_war_spend: float = 0.10
    k_subsidy: float = 0.06
    k_san_leak: float = 0.04

    # --- Inflation ---
    pi_base: float = 0.35
    pi_target: float = 0.10
    k_pi_san: float = 0.25
    k_pi_war: float = 0.35
    k_pi_fisc: float = 0.20

    # --- Economic stress mapping to 0-1 ---
    a_inf: float = 4.0
    a_fisc: float = 3.0
    a_war: float = 2.0

    # --- Repression capacity proxy ---
    r0: float = 0.9
    r_w: float = 0.3

    # --- Protest dynamics ---
    k_mob: float = 0.08
    k_demob: float = 0.10
    b1: float = 2.5
    b2: float = 1.2
    b3: float = 1.0
    b4: float = 0.8

```

```

b5: float = 2.2
c1: float = 2.0
c2: float = 1.0
c3: float = 0.6

# --- Morale dynamics ---
k_rally: float = 0.06
tau_rally: float = 26.0
k_fatigue: float = 0.05
k_price: float = 0.015
k_disp: float = 0.004
k_morale_recover: float = 0.03

# --- Stability dynamics ---
k_cons: float = 0.06
k_info: float = 0.02
k_loss_p: float = 0.08
k_loss_e: float = 0.05
k_loss_w: float = 0.03
k_rally_s: float = 0.05
k_stab_recover: float = 0.04

# --- Elite cohesion ---
k_elite_rally: float = 0.03
k_elite_cost: float = 0.05
k_elite_protest: float = 0.04
k_elite_recover: float = 0.02

# --- Loyalty ---
k_pay: float = 0.05
k_loss_l: float = 0.06
k_split: float = 0.05
k_loyal_recover: float = 0.015

# --- Information control ---
k_info_invest: float = 0.03
k_info_emerg: float = 0.04
k_info_deg_w: float = 0.03
k_info_decay_calm: float = 0.01

# --- Troop mobilization ---
k_troop_up: float = 0.05
k_troop_down: float = 0.04
k_troop_attr: float = 0.02

# --- Displacement (millions) ---
k_disp_w: float = 0.05
k_disp_p: float = 0.03
k_disp_e: float = 0.01
k_return: float = 0.03

```

```

# --- Oil price shock (index) ---
oil_price_base: float = 1.0
k_price_shock: float = 0.5

def scenario_exog(t_week: int, scenario: str) -> Tuple[float, float]:
    """Zwraca (W, San) w [0,1]. Profil scenariusza jest badawczy, nie
    operacyjny."""
    if scenario == "szybka_wojna":
        W = 1.0 if t_week < 8 else (0.5 if t_week < 12 else 0.1)
        San = 0.7 if t_week < 26 else 0.5
    elif scenario == "dlugotrwała_wojna":
        W = 0.8 if t_week < 52 else 0.6
        San = 0.85
    elif scenario == "impas":
        W = 0.5 + 0.15 * math.sin(2 * math.pi * t_week / 26)
        San = 0.75
    elif scenario == "eskalacja_regionalna":
        W = 0.9 if t_week < 26 else 0.7
        San = 0.9
    else:
        W = 0.0
        San = 0.3
    return clip(W), clip(San)

def step(state: Dict[str, float], t_week: int, p: Params, scenario: str) ->
Dict[str, float]:
    W, San = scenario_exog(t_week, scenario)

    # Oil price index (stylized global shock)
    OilPrice = p.oil_price_base * (1.0 + p.k_price_shock * W)

    # Resource revenue index
    Rev = p.oil_export_cap * (1.0 - p.k_san_oil * San) * (1.0 - p.k_war_oil
    * W) * OilPrice
    Rev = max(0.0, Rev)

    # Inflation (annual), CPI weekly update
    F = state["Fiscal"]
    pi = p.pi_base + p.k_pi_san * San + p.k_pi_war * W + p.k_pi_fisc *
    max(0.0, (0.5 - F))
    pi = max(0.0, pi)
    CPI = state["CPI"] * (1.0 + pi / 52.0)

    # Economic stress (0-1)
    Estress = sigmoid(p.a_inf * (pi - p.pi_target) + p.a_fisc * (0.5 - F) +
    p.a_war * W)

    # Calm factor (used for recovery terms)
    calm = (1.0 - W) * (1.0 - state["Protest"])

```



```

# Repression capacity proxy
E = state["Elite"]
L = state["Loyal"]
I = state["Info"]
Repr = clip((p.r0 * L * E + p.r_w * W) * I)

# Morale
M = state["Morale"]
D = state["Displaced"]
rally_term = p.k_rally * W * math.exp(-t_week / p.tau_rally)
M_next = clip(
    M
    + rally_term
    - p.k_fatigue * (W + Estress)
    - p.k_price * pi
    - p.k_disp * D
    + p.k_morale_recover * calm * (1.0 - M)
)

# Fiscal space
inflow = p.k_rev * Rev
outflow = p.k_war_spend * W + p.k_subsidy * (1.0 - M_next) +
p.k_san_leak * San
F_next = clip(F + inflow - outflow + 0.01 * calm * (1.0 - F))

# Information control
I_next = clip(
    I
    + p.k_info_invest * F_next
    + p.k_info_emerg * (W + state["Protest"])
    - p.k_info_deg_w * W
    - p.k_info_decay_calm * calm * I
)

# Protest stock
P = state["Protest"]
mobilize = p.k_mob * sigmoid(
    p.b1 * Estress
    + p.b2 * (1.0 - M_next)
    + p.b3 * (1.0 - state["Stab"])
    + p.b4 * (1.0 - I_next)
    - p.b5 * Repr
)
demobilize = p.k_demob * sigmoid(p.c1 * Repr + p.c2 * W + p.c3 * P) +
0.02 * calm * P
P_next = clip(P + mobilize - demobilize)

# Elite cohesion
E_next = clip(
    E

```

```

    + p.k_elite_rally * W
    - p.k_elite_cost * (Estress + W)
    - p.k_elite_protest * P_next
    + p.k_elite_recover * calm * (1.0 - E)
)

# Loyalty
L_next = clip(
    L
    + p.k_pay * F_next
    - p.k_loss_l * (W + Estress)
    - p.k_split * (1.0 - E_next)
    + p.k_loyal_recover * calm * (1.0 - L)
)

# Stability
S = state["Stab"]
rally_s = p.k_rally_s * W * math.exp(-t_week / p.tau_rally)
S_next = clip(
    S
    + p.k_cons * (E_next * L_next * F_next)
    + p.k_info * I_next
    + rally_s
    - p.k_loss_p * P_next
    - p.k_loss_e * Estress
    - p.k_loss_w * W
    + p.k_stab_recover * calm * (1.0 - S)
)

# Troop mobilization index
T = state["Troops"]
T_next = clip(
    T
    + p.k_troop_up * W * (1.0 - T)
    - p.k_troop_down * (1.0 - W) * T
    - p.k_troop_attr * W * T
)

# Displacement (millions)
new_disp = p.k_disp_w * W + p.k_disp_p * P_next * Repr + p.k_disp_e *
Estress
returns = p.k_return * (1.0 - W) * (1.0 - P_next) * state["Displaced"]
D_next = max(0.0, state["Displaced"] + new_disp - returns)

return {
    "Stab": S_next,
    "Elite": E_next,
    "Loyal": L_next,
    "Info": I_next,
    "Morale": M_next,
    "Protest": P_next,

```

```

        "Fiscal": F_next,
        "CPI": CPI,
        "Infl": pi,
        "EStress": Estress,
        "Repr": Repr,
        "Troops": T_next,
        "Displaced": D_next,
        "War": W,
        "Sanctions": San,
        "OilPrice": OilPrice,
        "OilRevIndex": Rev,
    }

def simulate(
    p: Params,
    scenario: str,
    years: int = 3,
    seed: int = 1,
    init: Optional[Dict[str, float]] = None,
) -> pd.DataFrame:
    # seed zostawiamy na przyszłe rozszerzenia (np. szum AR(1) w W/San lub
    # komponent losowy w represjach)
    steps = int(years * 52)
    if init is None:
        state = {
            "Stab": 0.60,
            "Elite": 0.65,
            "Loyal": 0.75,
            "Info": 0.70,
            "Morale": 0.55,
            "Protest": 0.20,
            "Fiscal": 0.45,
            "CPI": 100.0,
            "Troops": 0.20,
            "Displaced": 0.0,
        }
    else:
        state = init.copy()

    rows: List[Dict[str, float]] = []
    for t in range(steps):
        state = step(state, t, p, scenario)
        rows.append({"week": t, "year": t / 52.0, **state})
    return pd.DataFrame(rows)

def compute_metrics(
    df: pd.DataFrame,
    years: int,
    stab_crit: float = 0.30,

```

```

    elite_crit: float = 0.35,
    elite_streak_weeks: int = 4,
) -> Dict[str, float]:
    """
    Zwraca metryki numeryczne bez NaN (ważne dla SALib).
    Tcrit: jeśli nie osiągnięto progu, zwracamy max_weeks+1.
    elite_fracture_prob: tu jako 0/1 (do uśredniania po replikacjach).
    """

    max_weeks = int(years * 52)

    idx = df.index[df["Stab"] < stab_crit]
    t_crit = int(idx[0]) if len(idx) > 0 else (max_weeks + 1)

    streak = 0
    fracture = 0.0
    for val in df["Elite"].values:
        if val < elite_crit:
            streak += 1
            if streak >= elite_streak_weeks:
                fracture = 1.0
                break
        else:
            streak = 0

    return {
        "Tcrit": float(t_crit),
        "elite_fracture_prob": float(fracture),
        "avg_loyal": float(df["Loyal"].mean()),
        "peak_protest": float(df["Protest"].max()),
        "end_displaced_m": float(df["Displaced"].iloc[-1]),
    }

```

Interfejs wymagany przez Ciebie: `run_model(params, seed)`

To jest cienka warstwa, która bierze słownik parametrów (tylko te 20 analizowane), nadpisuje bazowe `Params()` i zwraca metryki.

```

# model_interface.py
# -*- coding: utf-8 -*-

from __future__ import annotations

from dataclasses import asdict
from typing import Dict, Any, List
import numpy as np

from sd_iran_model import Params, simulate, compute_metrics

GSA_PARAM_NAMES: List[str] = [

```

```

    "k_san_oil", "k_war_oil",
    "k_pi_san", "k_pi_war", "pi_base",
    "r0",
    "k_mob", "k_demob",
    "k_loss_p", "k_cons",
    "k_pay", "k_loss_l", "k_split",
    "k_elite_cost", "k_elite_protest",
    "k_info_emerg", "k_info",
    "k_rev", "k_war_spend",
    "k_rally_s",
]

def run_model(
    params: Dict[str, float],
    seed: int,
    scenario: str,
    years: int,
    stab_crit: float,
    elite_crit: float,
    elite_streak_weeks: int,
) -> Dict[str, float]:
    """
    Wymagany interfejs: run_model(params, seed) -> metryki.
    Rozszerzamy o scenariusz/years/progi w argumentach, aby sterować
    konfiguracją z YAML.
    """
    base = Params()
    base_dict = asdict(base)

    # nadpisz jedynie parametry z listy
    for k, v in params.items():
        if k not in base_dict:
            raise KeyError(f"Unknown parameter '{k}'.")
        base_dict[k] = float(v)

    p = Params(**base_dict)
    df = simulate(p, scenario=scenario, years=years, seed=seed)
    met = compute_metrics(
        df,
        years=years,
        stab_crit=stab_crit,
        elite_crit=elite_crit,
        elite_streak_weeks=elite_streak_weeks,
    )
    return met

```

Procedura Morris w SALib: sampling, analiza, wykresy

Metoda Morris (elementary effects) jest polecana jako **tani screening** przy dużej liczbie parametrów i kosztownych modelach; SALib wspiera wariant podstawowy oraz trajektorie optymalne (Campolongo, Ruano). ¹⁵ Wyniki w SALib obejmują m.in. μ , μ^* , σ oraz μ^*_conf . ¹⁶

Ustawienia obliczeniowe (rekomendacje)

Dla $D=20$:

- **Morris „quick”**: trajektorie $r=20$, $\text{levels}=8 \rightarrow$ ok. $r \cdot (D+1)=420$ punktów.
- **Morris „balanced”**: $r=30$, $\text{levels}=8$, $\text{optimal_trajectories}=10$, $\text{local_optimization}=\text{True} \rightarrow$ ok. 630 punktów, lepsze pokrycie przestrzeni. ¹⁷
- Każdy punkt: **n_reps=3-5** (uśrednianie metryk, bo model jest stochastyczny). ¹

Skrypt: `run_morris.py`

```
# run_morris.py
# -*- coding: utf-8 -*-

from __future__ import annotations

import os
import json
from typing import Dict, List, Tuple
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import yaml
from tqdm import tqdm
from joblib import Parallel, delayed

from SALib.sample.morris import sample as morris_sample
from SALib.analyze.morris import analyze as morris_analyze

from model_interface import run_model, GSA_PARAM_NAMES
from gsa_common import (
    build_problem_unit_hypercube,
    transform_unit_samples_to_params,
    aggregate_replicates,
    ensure_dir,
    plot_morris_bar,
)

def eval_one_point(
    param_dict: Dict[str, float],
    base_seed: int,
    rep_seeds: List[int],
    cfg: Dict,
```

```

) -> Dict[str, float]:
    mets = []
    for s in rep_seeds:
        mets.append(
            run_model(
                param_dict,
                seed=s,
                scenario=cfg["scenario"],
                years=int(cfg["years"]),
                stab_crit=float(cfg["stab_crit"]),
                elite_crit=float(cfg["elite_crit"]),
                elite_streak_weeks=int(cfg["elite_streak_weeks"]),
            )
        )
    return aggregate_replicates(mets)

def main():
    with open("config.yaml", "r", encoding="utf-8") as f:
        cfg = yaml.safe_load(f)

    out_dir = ensure_dir(os.path.join("outputs", "morris"))
    problem = build_problem_unit_hypercube(GSA_PARAM_NAMES)

    # Morris sampling in unit cube [0,1]
    X_unit = morris_sample(
        problem,
        N=int(cfg["morris_trajectories"]),
        num_levels=int(cfg["morris_num_levels"]),
        optimal_trajectories=int(cfg["morris_optimal_trajectories"]),
        local_optimization=bool(cfg["morris_local_optimization"]),
        seed=int(cfg["seed_base"]),
    )

    # Transform to physical parameters with priors
    X_phys, dist_spec = transform_unit_samples_to_params(X_unit)

    # Prepare seeds per point (common random numbers strategy)
    n_reps = int(cfg["n_reps"])
    base_seed = int(cfg["seed_base"])
    stride = int(cfg["rep_seed_stride"])

    rep_seeds = [base_seed + stride * i for i in range(n_reps)]

    # Evaluate
    def _task(i: int):
        pdict = {name: float(X_phys[i, j]) for j, name in
            enumerate(GSA_PARAM_NAMES)}
        return eval_one_point(pdict, base_seed=base_seed,
            rep_seeds=rep_seeds, cfg=cfg)

```

```

    results = Parallel(n_jobs=int(cfg["n_jobs"]))(delayed(_task)(i) for i in
    tqdm(range(X_phys.shape[0])))

    Y = pd.DataFrame(results)
    Y.to_csv(os.path.join(out_dir, "morris_model_outputs.csv"), index=False)

    # Morris analysis: trzeba wykonać osobno dla każdej metryki (SALib
    oczekuje wektora Y) 18
    metrics = ["Tcrit_mean", "elite_fracture_prob_mean",
    "peak_protest_mean", "end_displaced_m_mean", "avg_loyal_mean"]
    analyses = {}

    for m in metrics:
        Si = morris_analyze(
            problem=problem,
            X=X_unit,          # UWAGA: analiza w przestrzeni próbkowania
            Y=Y[m].values,
            num_levels=int(cfg["morris_num_levels"]),
            num_resamples=1000, # CI na  $\mu^*$  (bootstrap) 19
            conf_level=0.95,
            seed=int(cfg["seed_base"]),
            print_to_console=False,
        )
        analyses[m] = {k: Si[k].tolist() if hasattr(Si[k], "tolist") else
        Si[k] for k in Si.keys()}

        # Wykresy
        plot_morris_bar(Si, GSA_PARAM_NAMES, title=f"Morris  $\mu^*$  dla {m}",
        out_png=os.path.join(out_dir, f"morris_mu_star_{m}.png"))

        with open(os.path.join(out_dir, "morris_indices.json"), "w",
        encoding="utf-8") as f:
            json.dump(analyses, f, ensure_ascii=False, indent=2)

        print(f"[OK] Morris: zapisano outputs do {out_dir}")

    if __name__ == "__main__":
        main()

```

Ważne uwagi: - SALib opisuje, że analiza Sobol/Morris operuje na *bounded inputs*; w tym kodzie Morris próbuje w [0,1] i my dopiero transformujemy do „fizycznych” priory rozkładów (nie jest to jedyna metoda, ale jest spójna i wygodna).

- Morris w SALib raportuje μ^* , σ i μ^*_conf ; interpretacyjnie: wysoki μ^* = duży wpływ, wysoka σ = nieliniowości / interakcje. ²⁰

Procedura Sobol w SALib: sampling Saltelli, indeksy i interakcje

Sobol to klasyczna metoda wariancyjna:

- S1 = wpływ „samodzielny”,

- ST = wpływ łączny (w tym interakcje),
- S2 = interakcje par (opcjonalnie). ²¹

W SALib liczba próbek jest determinowana przez N i D:

- bez S2: $N \cdot (D+2)$,
- z S2: $N \cdot (2D+2)$. ²²

Ustawienia obliczeniowe (oszczędzanie kosztu)

D=20:

- etap 1 (roboczy): N=512, calc_second_order=False → 512·22=11264 uruchomień; n_reps=3 → 33k symulacji
- etap 2 (produkcyjny): N=1024 lub 2048, calc_second_order=False; S2 tylko gdy potrzebujesz mapy interakcji i masz budżet
- etap 3 (interakcje): N=1024, calc_second_order=True → 1024·42=43008 uruchomień; to jest już poziom „HPC”. ²³

Dodatkowo: ponieważ konwergencja indeksów bywa wrażliwa na estymator i schemat próbkowania, warto testować stabilność wyników po rosnącym N. ²⁴

Skrypt: `run_sobol.py`

```
# run_sobol.py
# -*- coding: utf-8 -*-

from __future__ import annotations

import os
import json
from typing import Dict, List
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import yaml
from tqdm import tqdm
from joblib import Parallel, delayed

from SALib.sample.saltelli import sample as saltelli_sample
from SALib.analyze.sobol import analyze as sobol_analyze

from model_interface import run_model, GSA_PARAM_NAMES
from gsa_common import (
    build_problem_unit_hypercube,
    transform_unit_samples_to_params,
    aggregate_replicates,
    ensure_dir,
    plot_sobol_bars,
    plot_sobol_s2_heatmap,
)
```

```

def main():
    with open("config.yaml", "r", encoding="utf-8") as f:
        cfg = yaml.safe_load(f)

    out_dir = ensure_dir(os.path.join("outputs", "sobol"))
    problem = build_problem_unit_hypercube(GSA_PARAM_NAMES)

    N = int(cfg["sobol_N"])
    calc_second_order = bool(cfg["sobol_calc_second_order"])

    # Saltelli sampling in unit cube
    X_unit = saltelli_sample(
        problem,
        N,
        calc_second_order=calc_second_order,
        seed=int(cfg["seed_base"]),
    )

    # Transform to physical parameters
    X_phys, dist_spec = transform_unit_samples_to_params(X_unit)

    # Common random numbers: stały zestaw replik seedów dla wszystkich
    # punktów
    n_reps = int(cfg["n_reps"])
    base_seed = int(cfg["seed_base"])
    stride = int(cfg["rep_seed_stride"])
    rep_seeds = [base_seed + stride * i for i in range(n_reps)]

    def _task(i: int) -> Dict[str, float]:
        pdict = {name: float(X_phys[i, j]) for j, name in
        enumerate(GSA_PARAM_NAMES)}
        mets = []
        for s in rep_seeds:
            mets.append(
                run_model(
                    pdict,
                    seed=s,
                    scenario=cfg["scenario"],
                    years=int(cfg["years"]),
                    stab_crit=float(cfg["stab_crit"]),
                    elite_crit=float(cfg["elite_crit"]),
                    elite_streak_weeks=int(cfg["elite_streak_weeks"]),
                )
            )
        return aggregate_replicates(mets)

    results = Parallel(n_jobs=int(cfg["n_jobs"]))(delayed(_task)(i) for i in
    tqdm(range(X_phys.shape[0])))

```

```

Y = pd.DataFrame(results)
Y.to_csv(os.path.join(out_dir, "sobol_model_outputs.csv"), index=False)

metrics = ["Tcrit_mean", "elite_fracture_prob_mean",
"peak_protest_mean", "end_displaced_m_mean", "avg_loyal_mean"]
analyses = {}

for m in metrics:
    Si = sobol_analyze(
        problem,
        Y[m].values,
        calc_second_order=calc_second_order,
        num_resamples=int(cfg["sobol_num_resamples"]), # bootstrap CI

        conf_level=float(cfg["sobol_conf_level"]),
        print_to_console=False,
        parallel=False, # często bezpieczniej (mniej RAM); SALib ma
opcję parallel, ale bywa pamięciożerna 25
        seed=int(cfg["seed_base"]),
    )
    analyses[m] = Si.to_df() if hasattr(Si, "to_df") else Si # w nowych
wersjach SALib jest ResultDict
    # zapis CSV
    if hasattr(Si, "to_df"):
        dfs = Si.to_df()
        # dfs: [S1, ST, S2] w zależności od opcji
        for k, df_k in enumerate(dfs):
            df_k.to_csv(os.path.join(out_dir, f"sobol_{m}_part{k}.csv"),
index=True)

        # wykresy S1 / ST
        plot_sobol_bars(Si, GSA_PARAM_NAMES, title=f"Sobol S1/ST dla {m}",
out_png=os.path.join(out_dir, f"sobol_bars_{m}.png"))

        # interakcje 2D (opcjonalnie)
        if calc_second_order:
            plot_sobol_s2_heatmap(Si, GSA_PARAM_NAMES, title=f"Sobol S2
(interakcje) dla {m}", out_png=os.path.join(out_dir, f"sobol_s2_{m}.png"))

        with open(os.path.join(out_dir, "sobol_indices_summary.json"), "w",
encoding="utf-8") as f:
            json.dump(_safe_jsonify(analyses), f, ensure_ascii=False, indent=2)

    print(f"[OK] Sobol: zapisano outputs do {out_dir}")

def _safe_jsonify(obj):
    # minimalna funkcja, by JSON nie wysypał się na np.float64
    if isinstance(obj, (str, int, float, bool)) or obj is None:
        return obj
    if isinstance(obj, np.generic):

```

```

        return obj.item()
    if isinstance(obj, dict):
        return {k: _safe_jsonify(v) for k, v in obj.items()}
    if isinstance(obj, list):
        return [_safe_jsonify(x) for x in obj]
    return str(obj)

if __name__ == "__main__":
    main()

```

Walidacja, diagnostyka i stabilizacja wyników (konwergencja, bootstrap, sanity checks)

Ponieważ Sobol jest „convergence-driven” (niebezpieczne indeksy są zwyczajnie błędne), część diagnostyczna jest obowiązkowa. ²⁶ SALib wprost wspiera bootstrap CI dla Sobola i Morrisa przez `num_resamples` i `conf_level`. ²⁷

Diagnostyka, którą warto wdrożyć „z automatu”

1. **Konwergencja po N:** uruchom Sobol dla $N \in \{256, 512, 1024, 2048\}$ (bez S2), a potem policz:
 2. $\max |S1(N) - S1(2N)|$,
 3. $\max |ST(N) - ST(2N)|$,
i wykres „stabilizacji indeksów”. W praktyce często widać, które parametry „dochodzą szybko”, a które wymagają większego N. ²⁸
4. **Bootstrap CI:** domyślnie 1000 resamples jest rozsądnym startem (SALib tak opisuje interfejs). ²⁹
5. **Sanity checks** (ważne dla modeli polityczno-ekonomicznych):
 6. $S1 \geq 0$ (czasem estymatory dają lekko ujemne wartości przez szum – wtedy interpretuj jako 0 w sensie „brak wpływu”),
 7. $\Sigma S1 \leq 1$ (z grubsza), $\Sigma ST \geq 1$ (z grubsza). ³⁰
 8. Jeżeli $ST \gg S1$ dla parametru $\rightarrow$ model ma silne interakcje z innymi wejściami (to jest ważna informacja o „kaskadach”). ³¹
9. **Diagnostyka szumu stochastycznego:**
10. porównaj wyniki dla $n_reps \in \{1, 3, 5, 10\}$;
11. policz wariancję między replikami dla tego samego punktu parametrów (to jest „intrinsic noise”). Stochastyczne symulatory wymagają tego kroku – bez niego Sobol może mierzyć głównie szum, a nie wpływ parametrów. ³²
12. **Common random numbers:** utrzymanie stałej listy seedów dla wszystkich punktów parametrów zwykle obniża wariancję estymacji efektów (w praktyce „wyrównuje szum tła”). Jest to zgodne z motywacją prac o GSA dla symulatorów losowych. ¹

Kod pomocniczy: `gsa_common.py`

W tym module trzymamy: definicję problemu SALib, transformację rozkładów, agregację replik, wykresy.

```
# gsa_common.py
# -*- coding: utf-8 -*-

from __future__ import annotations

import os
from dataclasses import dataclass
from typing import Dict, List, Tuple, Any
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import beta as beta_dist
from scipy.stats import truncnorm

def ensure_dir(path: str) -> str:
    os.makedirs(path, exist_ok=True)
    return path

def build_problem_unit_hypercube(param_names: List[str]) -> Dict[str, Any]:
    return {
        "num_vars": len(param_names),
        "names": list(param_names),
        "bounds": [[0.0, 1.0] for _ in param_names], # unit cube
    }

@dataclass(frozen=True)
class DistSpec:
    kind: str
    lo: float
    hi: float
    args: Tuple[float, ...]

def _scale(lo: float, hi: float, x01: np.ndarray) -> np.ndarray:
    return lo + (hi - lo) * x01

def _inv_scale(lo: float, hi: float, x: np.ndarray) -> np.ndarray:
    x01 = (x - lo) / (hi - lo)
    return np.clip(x01, 0.0, 1.0)

def transform_unit_samples_to_params(X_unit: np.ndarray) ->
```

```

Tuple[np.ndarray, Dict[str, DistSpec]]:
    """
    X_unit: (n, D) w [0,1]. Zwraca X_phys o tych samych wymiarach.
    Rozkłady zgodne z tabelą priory.
    """

    # Specyfikacje rozkładów: nazwa -> DistSpec(kind, lo, hi, args)
    dist_spec: Dict[str, DistSpec] = {
        "k_san_oil": DistSpec("beta", 0.20, 0.90, (5.0, 3.0)),
        "k_war_oil": DistSpec("beta", 0.30, 0.95, (4.0, 4.0)),
        "k_pi_san": DistSpec("truncnorm", 0.00, 0.50, (0.25, 0.06)),
        "k_pi_war": DistSpec("truncnorm", 0.00, 0.60, (0.35, 0.08)),
        "pi_base": DistSpec("truncnorm", 0.10, 0.80, (0.35, 0.10)),
        "r0": DistSpec("truncnorm", 0.50, 1.20, (0.90, 0.10)),
        "k_mob": DistSpec("truncnorm", 0.02, 0.15, (0.08, 0.02)),
        "k_demob": DistSpec("truncnorm", 0.02, 0.20, (0.10, 0.03)),
        "k_loss_p": DistSpec("truncnorm", 0.02, 0.20, (0.08, 0.03)),
        "k_cons": DistSpec("truncnorm", 0.01, 0.15, (0.06, 0.02)),
        "k_pay": DistSpec("truncnorm", 0.00, 0.18, (0.05, 0.03)),
        "k_loss_l": DistSpec("truncnorm", 0.00, 0.25, (0.06, 0.03)),
        "k_split": DistSpec("truncnorm", 0.00, 0.20, (0.05, 0.03)),
        "k_elite_cost": DistSpec("truncnorm", 0.00, 0.20, (0.05, 0.03)),
        "k_elite_protest": DistSpec("truncnorm", 0.00, 0.20, (0.04, 0.03)),
        "k_info_emerg": DistSpec("truncnorm", 0.00, 0.20, (0.04, 0.03)),
        "k_info": DistSpec("truncnorm", 0.00, 0.08, (0.02, 0.015)),
        "k_rev": DistSpec("truncnorm", 0.02, 0.30, (0.12, 0.06)),
        "k_war_spend": DistSpec("truncnorm", 0.02, 0.25, (0.10, 0.05)),
        "k_rally_s": DistSpec("truncnorm", 0.00, 0.20, (0.05, 0.05)),
    }

    # Nazwy muszą być w tej samej kolejności co próbki
    # X_unit kolumny odpowiadają problem['names'] ustawionym w run_*.py
    # Wykonujemy transformację kolumnowo
    # Zakładamy, że X_unit jest w [0,1]
    X_phys = np.zeros_like(X_unit, dtype=float)

    # problem['names'] jest dostępne w run_*.py, ale tu zakładamy, że jest
    # identyczne z dist_spec keys order
    # dlatego run_*.py przekazuje paramy w tej samej kolejności.
    from model_interface import GSA_PARAM_NAMES
    for j, name in enumerate(GSA_PARAM_NAMES):
        u = np.clip(X_unit[:, j], 1e-12, 1.0 - 1e-12)
        spec = dist_spec[name]

        if spec.kind == "beta":
            a, b = spec.args
            x01 = beta_dist.ppf(u, a, b)
            X_phys[:, j] = _scale(spec.lo, spec.hi, x01)

        elif spec.kind == "truncnorm":
            mu, sigma = spec.args
            # truncnorm w SciPy parametr ustandaryzowany

```

```

        a = (spec.lo - mu) / sigma
        b = (spec.hi - mu) / sigma
        X_phys[:, j] = truncnorm.ppf(u, a, b, loc=mu, scale=sigma)

    else:
        raise ValueError(f"Unknown dist kind: {spec.kind}")

    return X_phys, dist_spec

def aggregate_replicates(metrics_list: List[Dict[str, float]]) -> Dict[str, float]:
    """
    Zwraca mean i std po replikacjach, dla każdej metryki.
    To jest ważne dla stochastycznego modelu (GSA na średniej).
    """
    keys = metrics_list[0].keys()
    out = {}
    for k in keys:
        vals = np.array([m[k] for m in metrics_list], dtype=float)
        out[f"{k}_mean"] = float(vals.mean())
        out[f"{k}_std"] = float(vals.std(ddof=1)) if len(vals) > 1 else 0.0
    return out

def plot_morris_bar(Si: Dict[str, np.ndarray], names: List[str], title: str,
out_png: str) -> None:
    mu_star = np.array(Si["mu_star"])
    order = np.argsort(-mu_star)

    plt.figure(figsize=(10, 6))
    plt.bar(range(len(names)), mu_star[order])
    plt.xticks(range(len(names)), [names[i] for i in order], rotation=75,
ha="right")
    plt.title(title)
    plt.ylabel("mu_star (Morris)")
    plt.tight_layout()
    plt.savefig(out_png, dpi=200)
    plt.close()

def plot_sobol_bars(Si, names: List[str], title: str, out_png: str) -> None:
    # SALib zwraca ResultDict z polami 'S1', 'ST' (+ _conf)
    S1 = np.array(Si["S1"])
    ST = np.array(Si["ST"])
    order = np.argsort(-ST)

    plt.figure(figsize=(10, 6))
    plt.bar(range(len(names)), ST[order], label="ST")
    plt.bar(range(len(names)), S1[order], label="S1")
    plt.xticks(range(len(names)), [names[i] for i in order], rotation=75,

```

```

ha="right")
    plt.title(title)
    plt.ylabel("Sobol index")
    plt.legend()
    plt.tight_layout()
    plt.savefig(out_png, dpi=200)
    plt.close()

def plot_sobol_s2_heatmap(Si, names: List[str], title: str, out_png: str) ->
None:
    S2 = Si["S2"] # macierz DxD
    D = len(names)
    mat = np.array(S2, dtype=float)

    plt.figure(figsize=(8, 7))
    plt.imshow(mat, aspect="auto")
    plt.colorbar(label="S2")
    plt.xticks(range(D), names, rotation=90)
    plt.yticks(range(D), names)
    plt.title(title)
    plt.tight_layout()
    plt.savefig(out_png, dpi=220)
    plt.close()

```

Format wyników, notebook i interpretacja (przykładowa)

Pliki wyjściowe

Procedura zapisuje:

- `outputs/morris/morris_model_outputs.csv` – metryki (średnie i odchylenia) dla wszystkich punktów Morris
- `outputs/morris/morris_indices.json` – wyniki analizy Morris per metryka
- `outputs/morris/morris_mu_star_<metric>.png` – barplot μ^* (Morris)
- `outputs/sobol/sobol_model_outputs.csv` – metryki dla Saltelli
- `outputs/sobol/sobol_<metric>_part*.csv` – tabele S1/ST (+ S2 gdy włączone)
- `outputs/sobol/sobolBars_<metric>.png` – barplot S1/ST
- `outputs/sobol/sobol_s2_<metric>.png` – heatmapa interakcji (opcjonalnie)

Przykładowe reguły interpretacyjne (na końcu raportu wróć do rekomendacji kalibracji)

- Jeśli dla `Tcrit_mean` parametry fiskalne/inflacyjne mają wysokie ST → model jest typowo „ekonomicznie sterowany” (spirala inflacja→stres→protest→stabilność), co jest spójne z tezą modelu o presji systemowej i kaskadach.

- Jeśli `k_info_emerg` ma wysokie S1 dla `elite_fracture_prob_mean` lub `peak_protest_mean` → mechanizmy kontroli informacji są centralne (Iran ma jedno z najbardziej restrykcyjnych warunków online, a telemetryczne dane shutdownów są dostępne).
- Jeśli $ST \gg S1$ dla parametrów lojalności/elit (`k_split`, `k_elite_cost`, `k_pay`) → to znaczy, że wpływ idzie głównie przez **interakcje** (np. $\text{fiskus} \times \text{protest} \times \text{elity}$), co bardzo pasuje do „dualizmu” autokracji: konflikt masa-reżim oraz konflikt elitarno-koalicyjny.

Szablon notebooka Jupyter

SALib ma interfejs `ProblemSpec` i chaining, ale przy stochastycznym modelu i wielowymiarowości zwykle wygodniej jest kontrolować pętle samemu. Mimo to warto znać `ProblemSpec` jako alternatywę.

Proponowany notebook `notebooks/00_template_gsa.ipynb`:

- sekcja 1: wczytaj config, ustaw `n_reps`
- sekcja 2: uruchom Morris (np. z `run_morris.py` jako import) i pokaż top-10 μ^*
- sekcja 3: uruchom Sobol dla $N=512$ i $N=1024$; porównaj stabilność indeksów (konwergencja)
- sekcja 4: heatmapa $S2$ dla jednej metryki (np. `Tcrit_mean`) tylko jeśli `calc_second_order=True`
- sekcja 5: zapis interpretacji i rekomendacji kalibracyjnych

Instrukcja uruchomienia krok po kroku oraz warianty oszczędzania kosztu

Uruchomienie lokalne

1. Utwórz środowisko:

```
python -m venv .venv
source .venv/bin/activate # (Windows: .venv\Scripts\activate)
pip install -r requirements.txt
```

2. Ustaw parametry w `config.yaml` (szczególnie: `years`, `scenario`, `n_reps`, `sobol_N`).

3. Morris (screening):

```
python run_morris.py
```

4. Sobol (dokładna dekompozycja):

```
python run_sobol.py
```

Gdy model jest „kosztowny”: 4 techniki redukcji kosztu

1. **Wyłącz S2** (`sobol_calc_second_order: false`). Zmniejsza liczbę uruchomień z $N \cdot (2D+2)$ do $N \cdot (D+2)$. ³
2. **Zrób Morris → redukcja wymiaru → Sobol**: Morris eliminuje parametry o niskim μ^* , a Sobol liczysz już na 8–12 parametrach. To klasyczna praktyka: screening przed wariancyjną GSA. ³⁵
3. **Uśrednianie po mniejszej liczbie replik**: start `n_reps=3`, dopiero na końcu podnieś do 10 w ostatniej iteracji. Dla stochastycznych symulatorów to kontroluje trade-off koszt/wiarygodność. ³²
4. **Surrogate / metamodel** (opcjonalnie, etap 2): dla stochastycznych symulatorów stosuje się podejścia z meta-modelami, np. by liczyć GSA na przybliżeniu funkcji średniej/kwantyli. ³⁶

HPC / parallelizacja

- joblib: łatwe równoległe mapowanie punktów parametrów (to jest „embarrassingly parallel”). ³⁷
- Dask: sensowny, jeśli chcesz iść w klastery lub pipeline danych (`map_partitions / delayed`). ³⁸
- Uwaga praktyczna: `sobol_analyze(parallel=True)` w SALib istnieje (wewnętrznie używa multiprocessing Pool), ale bywa pamięciożerne; często bezpieczniej zrównoleglić **ewaluację modelu** (joblib), a analizę zostawić pojedynczemu procesowi. ³⁹

Alternatywne testy wrażliwości i rekomendacje kalibracyjne dla Iranu

Alternatywne analizy wrażliwości (uzupełniające GSA)

1. **Lokalna wrażliwość** (finite differences wokół punktu bazowego): szybka, dobra do sanity check i do sprawdzenia, czy model zachowuje się monotonicznie lokalnie (nie zastępuje GSA).
2. **Scenariusze skrajne (stress tests)**: np. „najgorszy fiskus + najwyższa inflacja + najsilniejsze sankcje” vs „najlepszy fiskus + niska inflacja + słaba presja”. To jest spójne z Twoim podejściem „resilience vs load”. `filecite{turn0file0}`
3. **PRCC / korelacje rangowe** na próbkach LHS: szybkie i często stabilne przy szumie, ale mniej formalne niż Sobol.
4. **Metody moment-independent (PAWN / Delta)**, dostępne w SALib: bywają bardziej odporne na nieliniowości i wielomodalność wyjść niż S1/Sobol (to opcja, gdy metryki są mocno nieliniowe). ⁴⁰

W pakiecie można dodać: - `run_local_sa.py` – lokalne pochodne numeryczne,
- `run_extremes.py` – generator skrajnych scenariuszy.

Priorytetowe parametry do dalszej kalibracji na danych irańskich

Zgodnie z Twoją dokumentacją modelu, priorytetem kalibracji są: elastyczności ropy, pass-through inflacji, represja/mobilizacja, kontrola informacji. `filecite{turn0file0}` W praktyce, biorąc pod uwagę „mierność” zmiennych latentnych E i L (trudne do bezpośredniego pomiaru), rekomenduję taki porządek:

1. **Inflacja i CPI (`pi_base`, `k_pi_san`, `k_pi_war`)**
Dane: WDI (FP.CPI.TOTL.ZG) i definicje wskaźnika; alternatywy: IMF oraz źródła krajowe. ⁴¹
2. **Dochody rentierskie i ropa (`k_san_oil`, `k_war_oil`, `k_rev`)**
Dane: „oil rents (% GDP)” w WDI; (dla ilości/rynków) uzupełnij IEA. ⁴²

3. Protesty (k_mob, k_demob, k_loss_p)

Dane zdarzeniowe: Entity["organization", "Armed Conflict Location & Event Data Project", "acled conflict database"] (np. przez HDX, agregacja tygodniowa). ⁴³

4. Kontrola informacji (k_info_emerg, k_info)

Dane jakościowe i zdarzeniowe shutdownów: Freedom House ⁴⁴ (Freedom on the Net Iran), telemetryczne potwierdzenia: Cloudflare ⁴⁵ (Radar / blog o shutdownach) oraz monitoring: NetBlocks ⁴⁶ . ⁴⁷

5. Sankcje (wejścia scenariuszowe San(t) i mapowanie na parametry)

Źródła oficjalne: Office of Foreign Assets Control ⁴⁸ (program Iran) oraz EUR-Lex ⁴⁹ (restrictive measures). ⁵⁰

6. Migracje/przesiedlenia (k_disp_*, k_return) – jeśli wchodzi do kalibracji

Źródła: UNHCR ⁵¹ (country/annual reports), a dla sytuacji kryzysowych także United Nations Office for the Coordination of Humanitarian Affairs ⁵² (ReliefWeb / OCHA updates). ⁵³

Źródła danych do pobrania (checklista)

- Makro: World Bank ⁵⁴ WDI (inflacja FP.CPI.TOTL.ZG; oil rents NY.GDP.PETR.RT.ZS). ⁵⁵
- Raporty krajowe: International Monetary Fund ⁵⁶ (strona kraju, dokumenty Article IV / raporty). ⁵⁷
- Energia i szoki: International Energy Agency ⁵⁸ (factsheet Strait of Hormuz dla logiki szoków / ryzyka). ⁵⁹
- Protesty: ACLED (HDX). ⁴³
- Internet: Freedom House / NetBlocks / Cloudflare. ⁶⁰
- Uchodźcy: UNHCR (Iran Annual Results Report / portal danych). ⁶¹

Na marginesie: w Twoim formalizmie pojawia się też warstwa 27D/9R jako metadane do kodowania zdarzeń i niepewności; jeśli będziesz łączyć GSA z analizą zdarzeniową, możesz tę warstwę wykorzystać do jawnego tagowania pewności danych wejściowych. `filecite{turn0file2}`

Praktyczne rekomendacje kalibracyjne dla Iranu

1. **Najpierw „usztynij” podsystem gospodarczy** (inflacja/fiskus/ropa), bo to jest źródło stresu w Twoim modelu i jednocześnie ma najlepsze dane obserwowalne (WDI/IMF/CBI). Dopiero potem dopasowuj mobilizację P oraz parametry rdzenia (E/L).
2. **Zrób Morris na 20 parametrach**, obetnij do ~10–12 najsilniejszych (μ^*), a dopiero potem rób Sobol. To minimalizuje koszt i jest zgodne z rolą Morrisa jako metody przesiewowej. ⁶²
3. **Dla stochastyczności** ustaw „common random numbers” + `n_reps=3` jako standard; `n_reps=10` odpal tylko w końcowym raporcie (na zredukowanej liczbie parametrów). To jest realistyczny kompromis koszt/wiarygodność dla stochastycznych symulatorów. ³²
4. **Nie kalibruj E i L „wprost”** bez OSINT-owych proxy (rotacje, czystki, defekcje, sygnały lojalności). Traktuj je jako latent i dopasowuj przez zdolność modelu do odtworzenia reżimów (stabilizacja represyjna / druga fala protestów / pęknięcie elit). To jest zgodne z Twoją częścią teoretyczną o dualizmie autokracji i naturze zmiennych latentnych. `filecite{turn0file1}`
5. **Kontrola informacji** w Iranie ma mocne dane najbliższe obserwowalnym (shutdown events + telemetryczne miary ruchu), więc ten moduł warto realnie skalibrować, a nie tylko „ustawić”. ⁶³

¹ ⁴ ³² ³⁶ ⁴⁴ Global sensitivity analysis for stochastic simulators based ...
https://www.sciencedirect.com/science/article/pii/S0951832021003379?utm_source=chatgpt.com

² ⁶ ¹⁵ ¹⁷ SALib.sample.morris package - Read the Docs
https://salib.readthedocs.io/en/latest/api/SALib.sample.morris.html?utm_source=chatgpt.com

- 3 22 23 54 **SALib.sample.sobol - Read the Docs**
https://salib.readthedocs.io/en/latest/_modules/SALib/sample/sobol.html?utm_source=chatgpt.com
- 5 **World Development Indicators | DataBank**
https://databank.worldbank.org/data/reports.aspx?country=IRN%2CIRL&series=FP.CPI.TOTL.ZG&source=2&utm_source=chatgpt.com
- 7 39 **SALib.analyze.sobol**
https://salib.readthedocs.io/en/latest/_modules/SALib/analyze/sobol.html?utm_source=chatgpt.com
- 8 41 **Inflation, consumer prices (annual %) - Iran, Islamic Rep.**
https://data.worldbank.org/indicator/FP.CPI.TOTL.ZG?locations=IR&utm_source=chatgpt.com
- 9 33 47 56 60 **Iran: Freedom on the Net 2024 Country Report**
https://freedomhouse.org/country/iran/freedom-net/2024?utm_source=chatgpt.com
- 10 58 63 **What we know about Iran's Internet shutdown**
https://blog.cloudflare.com/iran-protests-internet-shutdown/?utm_source=chatgpt.com
- 11 **Global Sensitivity Analysis of Model Outputs**
https://r.meteo.uni.wroc.pl/web/packages/sensitivity/sensitivity.pdf?utm_source=chatgpt.com
- 12 13 40 **SALib · PyPI**
<https://pypi.org/project/SALib/>
- 14 37 **Embarrassingly parallel for loops - Joblib - Read the Docs**
https://joblib.readthedocs.io/en/latest/parallel.html?utm_source=chatgpt.com
- 16 20 **SALib.analyze.morris - Read the Docs**
https://salib.readthedocs.io/en/latest/_modules/SALib/analyze/morris.html?utm_source=chatgpt.com
- 18 48 **Compute Si for multiple outputs in parallel · Issue #41**
https://github.com/SALib/SALib/issues/41?utm_source=chatgpt.com
- 19 27 29 49 **SALib.analyze package — SALib's documentation**
https://salib.readthedocs.io/en/latest/api/SALib.analyze.html?utm_source=chatgpt.com
- 21 30 31 **Basics — SALib's documentation - Read the Docs**
https://salib.readthedocs.io/en/latest/user_guide/basics.html?utm_source=chatgpt.com
- 24 26 28 45 52 **Convergence, sampling and total order estimator effects on ...**
https://pmc.ncbi.nlm.nih.gov/articles/PMC11285933/?utm_source=chatgpt.com
- 25 **Memory error - parallel processing - Sobol analyze #140**
https://github.com/SALib/SALib/issues/140?utm_source=chatgpt.com
- 34 46 **SALib Interface Basics - Read the Docs**
https://salib.readthedocs.io/en/latest/user_guide/basics_with_interface.html?utm_source=chatgpt.com
- 35 62 **Elementary effects method**
https://en.wikipedia.org/wiki/Elementary_effects_method?utm_source=chatgpt.com
- 38 51 **Dask Best Practices — Dask documentation**
https://docs.dask.org/en/stable/best-practices.html?utm_source=chatgpt.com
- 42 **Oil rents (% of GDP) - Iran, Islamic Rep.**
https://data.worldbank.org/indicator/NY.GDP.PETR.RT.ZS?locations=IR&utm_source=chatgpt.com
- 43 **Iran - Conflict Events | Humanitarian Dataset | HDX**
https://data.humdata.org/dataset/iran-acled-conflict-data?utm_source=chatgpt.com

50 **Iran Sanctions | Office of Foreign Assets Control - Treasury**

https://ofac.treasury.gov/sanctions-programs-and-country-information/iran-sanctions?utm_source=chatgpt.com

53 61 **Annual Results Report - 2024 Iran**

https://www.unhcr.org/sites/default/files/2025-06/Iran%20ARR%202024.pdf?utm_source=chatgpt.com

55 **Inflation - Glossary | DataBank - World Bank**

https://databank.worldbank.org/metadataglossary/world-development-indicators/series/FP.CPI.TOTL.ZG?utm_source=chatgpt.com

57 **Islamic Republic of Iran and the IMF**

https://www.imf.org/en/countries/irn?utm_source=chatgpt.com

59 **Strait of Hormuz - About**

https://www.iea.org/about/oil-security-and-emergency-response/strait-of-hormuz?utm_source=chatgpt.com